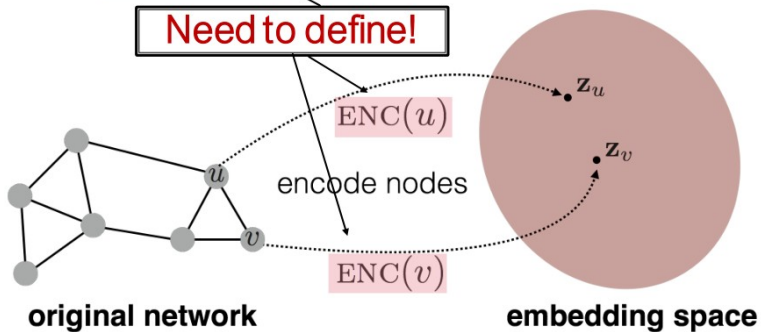


Случайные блуждания (Random Walks)

Энкодер и декодер в задаче нахождения эмбединга

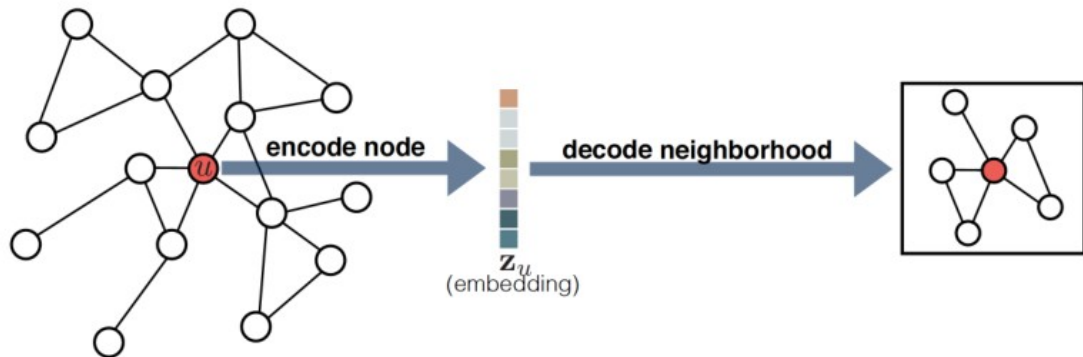
Goal: $\text{similarity}(u, v)$ in the original network $\approx \mathbf{z}_v^T \mathbf{z}_u$ Similarity of the embedding



$$\text{ENC} : \mathcal{V} \rightarrow \mathbb{R}^d,$$

$$\text{ENC}(v) = \mathbf{Z}[v],$$

Энкодер – кодирование глобальной и локальной информации о ноде



Декодер – восстановление информации о локальном окружении нода

$$\text{DEC} : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^+.$$

$$\text{DEC}(\text{ENC}(u), \text{ENC}(v)) = \text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \approx \mathbf{S}[u, v].$$

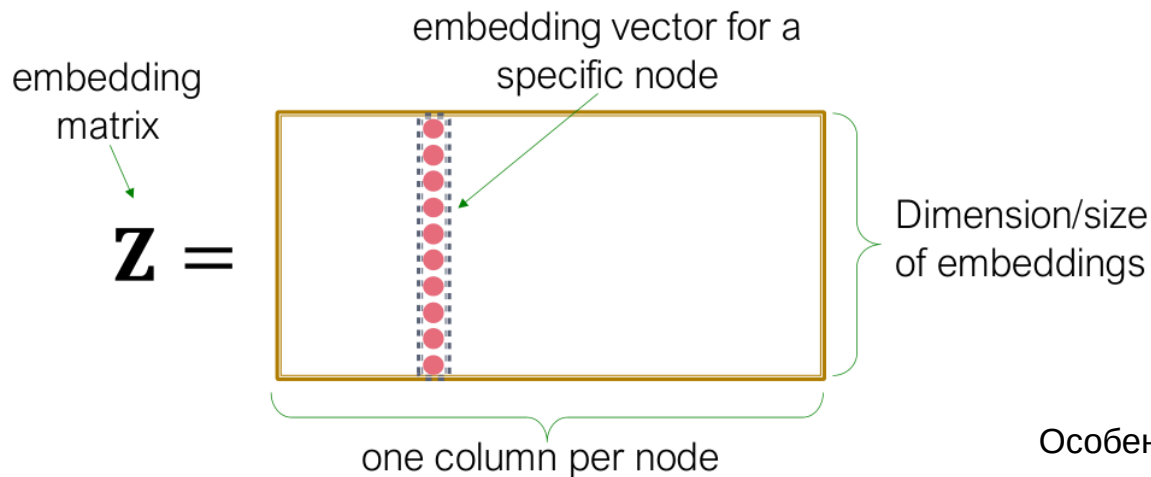
Проще всего считать что декодер восстанавливает информацию о близости двух нод $\mathbf{S}[u, v]$

Задача оптимизации с учителем:

$$\mathcal{L} = \sum_{(u, v) \in \mathcal{D}} \ell(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v), \mathbf{S}[u, v]) \rightarrow \min$$

Оптимальный эмбедингг (поверхностное кодирование)

Simplest encoding approach: **encoder is just an embedding-lookup**



Достоинства:

Можно сделать оптимальным для каждой задачи

Недостатки:

- Неприменим для новых нод
- Не учитывает общие особенности (топологию) нод
- Не использует внутренние параметры нод
- Требуется много ресурсов для обучения – число параметров линейно по числу нод

Особенности методов поверхностного кодирования

Method	Decoder	Similarity measure	Loss function
Lap. Eigenmaps	$\ \mathbf{z}_u - \mathbf{z}_v\ _2^2$	general	$\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]$
Graph Fact.	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v]$	$\ \text{DEC}(\mathbf{z}_u, \mathbf{z}_v) - \mathbf{S}[u, v]\ _2^2$
GraRep	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v], \dots, \mathbf{A}^k[u, v]$	$\ \text{DEC}(\mathbf{z}_u, \mathbf{z}_v) - \mathbf{S}[u, v]\ _2^2$
HOPE	$\mathbf{z}_u^\top \mathbf{z}_v$	general	$\ \text{DEC}(\mathbf{z}_u, \mathbf{z}_v) - \mathbf{S}[u, v]\ _2^2$
DeepWalk	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in \mathcal{V}} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p_{\mathcal{G}}(v u)$	$-\mathbf{S}[u, v] \log(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v))$
node2vec	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in \mathcal{V}} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p_{\mathcal{G}}(v u)$ (biased)	$-\mathbf{S}[u, v] \log(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v))$

Эмбе́ддинг для топологии

Для получения универсального эмбе́ддинга он должен описывать только топологию (структуру) графа с точки зрения кодируемой ноды и поэтому **не должен**:

- включать в себя номера нод
- включать в себя атрибуты нод
- зависеть от решаемой задачи

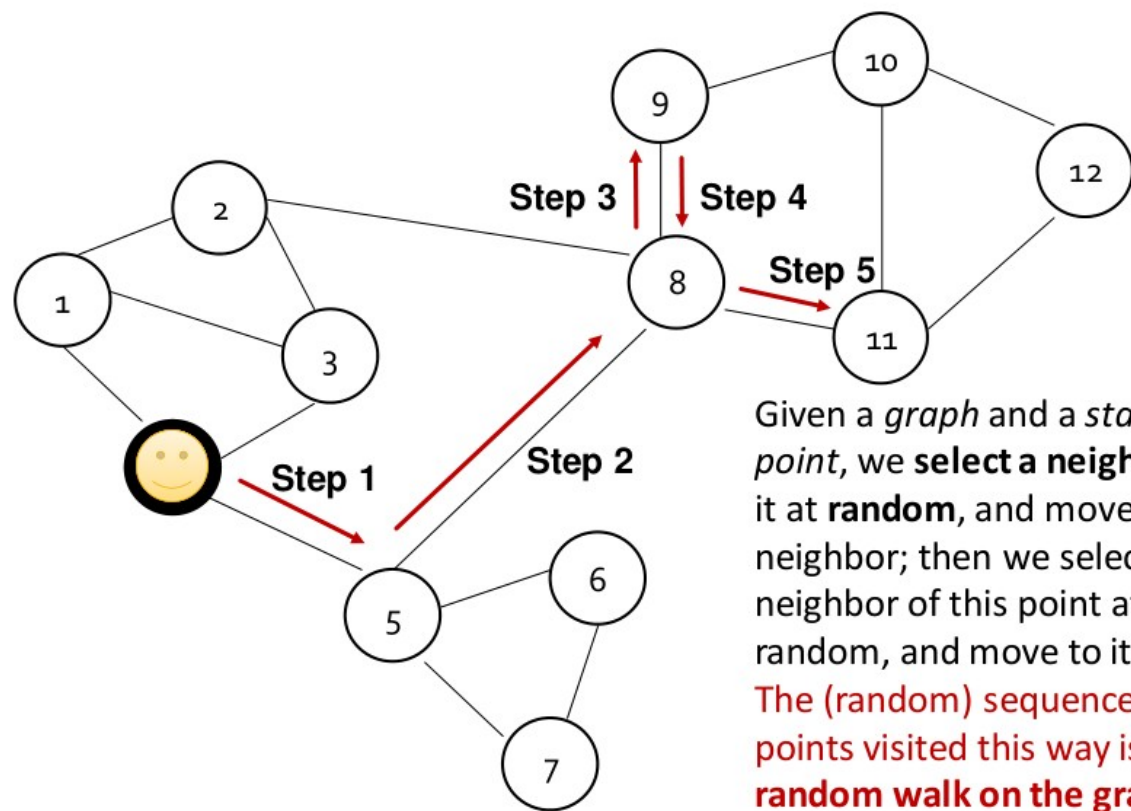
Обозначения

- **Vector** $\mathbf{z}_u$:
 - The embedding of node u (what we aim to find).
 - **Probability** $P(v | \mathbf{z}_u)$:  Our model prediction based on $\mathbf{z}_u$
 - The **(predicted) probability** of visiting node v on random walks starting from node u .
-

Non-linear functions used to produce predicted **probabilities**

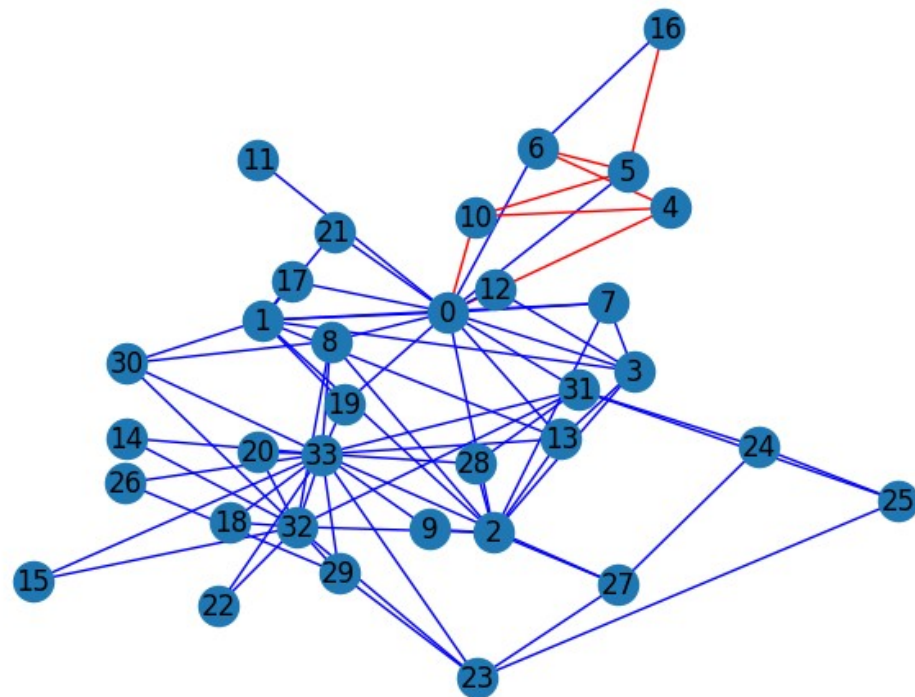
- **Softmax** function:
 - Turns vector of K real values (model predictions) into K probabilities that sum to 1: $\sigma(\mathbf{z})[i] = \frac{e^{z[i]}}{\sum_{j=1}^K e^{z[j]}}$
- **Sigmoid** function:
 - S-shaped function that turns real values into the range of (0, 1).
Written as $S(x) = \frac{1}{1+e^{-x}}$.

Случайное блуждание



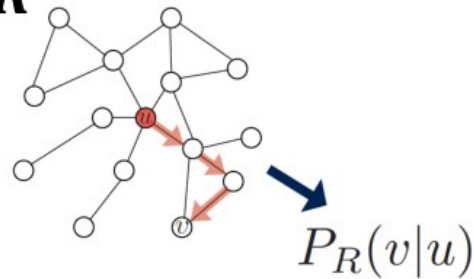
Случайное блуждание

```
import networkx as nx, matplotlib.pyplot as plt, numpy as np
G = nx.karate_club_graph()
pos = nx.kamada_kawai_layout(G)
node0, code_len = 16, 10
node, code = node0, [node0]
for itt in range(code_len):
    nodes = G[node]
    l = len(nodes)
    idxs = [i for i in nodes.keys()]
    idx = np.random.randint(len(idxs))
    node = idxs[idx]
    code.append(node)
c, k = [], 0
for u, v in G.edges:
    found = False
    for k in range(len(code)-1):
        if (code[k] == u and code[k+1] == v) or (code[k] == v and code[k+1] == u):
            found = True
            break
    if found: c.append('red')
    else: c.append('blue')
nx.draw(G, pos, with_labels=True, edge_color=c)
print(code)
```

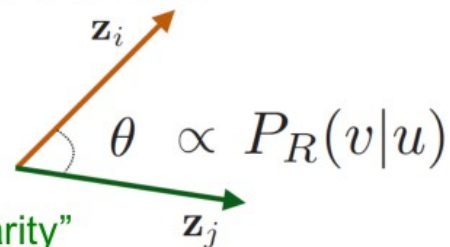


[16, 5, 6, 4, 10, 5, 6, 5, 10, 0, 4]

1. Estimate probability of visiting node v on a random walk starting from node u using some random walk strategy R



2. Optimize embeddings to encode these random walk statistics:



Similarity in embedding space (Here:
dot product= $\cos(\theta)$) encodes random walk “similarity”

1. **Expressivity:** Flexible stochastic definition of node similarity that incorporates both local and higher-order neighborhood information
Idea: if random walk starting from node u visits v with high probability, u and v are similar (high-order multi-hop information)
~~similar~~
close
2. **Efficiency:** Do not need to consider all node pairs when training; only need to consider pairs that co-occur on random walks

Случайное блуждание – формулировка задачи

- **Intuition:** Find embedding of nodes in d -dimensional space that preserves similarity
- **Idea:** Learn node embedding such that **nearby** nodes are close together in the network
- **Given a node u , how do we define nearby nodes?**
 - $N_R(u)$... neighbourhood of u obtained by some random walk strategy R

- Given $G = (V, E)$,
- Our goal is to learn a mapping $f: u \rightarrow \mathbb{R}^d$:
 $f(u) = \mathbf{z}_u$

- Log-likelihood objective:

$$\max_f \sum_{u \in V} \log P(N_R(u) | \mathbf{z}_u)$$

- $N_R(u)$ is the neighborhood of node u by strategy R
- **Given node u , we want to learn feature representations that are predictive of the nodes in its random walk neighborhood $N_R(u)$.**

Случайное блуждание – решение

1. Run **short fixed-length random walks** starting from each node u in the graph using some random walk strategy R .
2. For each node u collect $N_R(u)$, the multiset* of nodes visited on random walks starting from u .
3. Optimize embeddings according to: **Given node u , predict its neighbors $N_R(u)$.**

$$\max_f \sum_{u \in V} \log P(N_R(u) | \mathbf{z}_u) \Rightarrow \text{Maximum likelihood objective}$$

* $N_R(u)$ can have repeat elements since nodes can be visited multiple times on random walks

Equivalently,

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v | \mathbf{z}_u))$$

- **Intuition:** Optimize embeddings $\mathbf{z}_u$ to maximize the likelihood of random walk co-occurrences.
- **Parameterize $P(v | \mathbf{z}_u)$ using softmax:**

$$P(v | \mathbf{z}_u) = \frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}$$

Why softmax?

We want node v to be most similar to node u (out of all nodes n).

Intuition: $\sum_i \exp(x_i) \approx \max_i \exp(x_i)$

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log\left(\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}\right) \rightarrow \min$$

sum over all nodes u sum over nodes v seen on random walks starting from u predicted probability of u and v co-occurring on random walk

Случайное блуждание – более быстрое решение – негативный сэмплинг

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log\left(\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}\right)$$

Nested sum over nodes gives
 $O(|V|^2)$ complexity!

$$\log\left(\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}\right)$$

random distribution
over nodes

$$\approx \log\left(\sigma(\mathbf{z}_u^T \mathbf{z}_v)\right) - \sum_{i=1}^k \log\left(\sigma(\mathbf{z}_u^T \mathbf{z}_{n_i})\right), n_i \sim P_V$$

- Sample k negative nodes each with prob. proportional to its degree
- Two considerations for k (# negative samples):
 1. Higher k gives more robust estimates
 2. Higher k corresponds to higher bias on negative eventsIn practice $k = 5-20$.

Can negative sample be any node or only the nodes not on the walk? People often use any nodes (for efficiency). However, the most “correct” way is to use nodes not on the walk.

■ **Gradient Descent**: a simple way to minimize $\mathcal{L}$:

■ Initialize z_u at some randomized value for all nodes u .

■ Iterate until convergence:

■ For all u , compute the derivative $\frac{\partial \mathcal{L}}{\partial z_u}$.

η : learning rate

■ For all u , make a step in reverse direction of derivative: $z_u \leftarrow z_u - \eta \frac{\partial \mathcal{L}}{\partial z_u}$.

Stochastic Gradient Descent: Instead of evaluating gradients over all examples, evaluate it for each **individual** training example.

$$\log(\sigma(\mathbf{z}_u^T \mathbf{z}_v)) - \sum_{i=1}^k \log(\sigma(\mathbf{z}_u^T \mathbf{z}_{n_i}))$$

■ Initialize z_u at some randomized value for all nodes u .

■ Iterate until convergence: $\mathcal{L}^{(u)} = \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$

■ Sample a node u , for all v calculate the derivative $\frac{\partial \mathcal{L}^{(u)}}{\partial z_v}$.

■ For all v , update: $z_v \leftarrow z_v - \eta \frac{\partial \mathcal{L}^{(u)}}{\partial z_v}$.

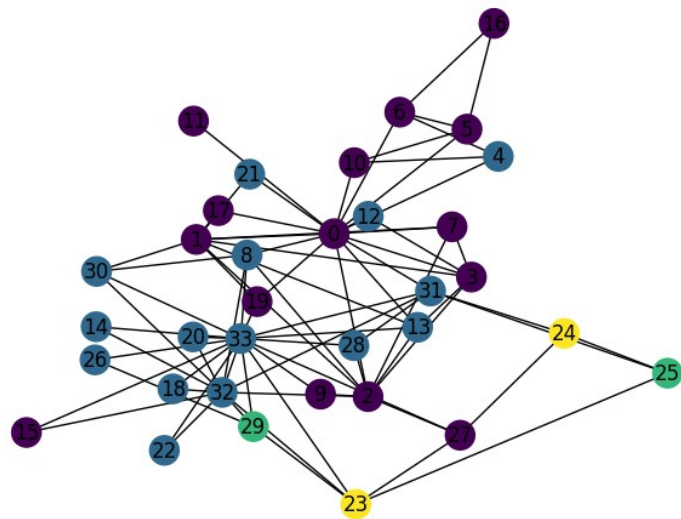
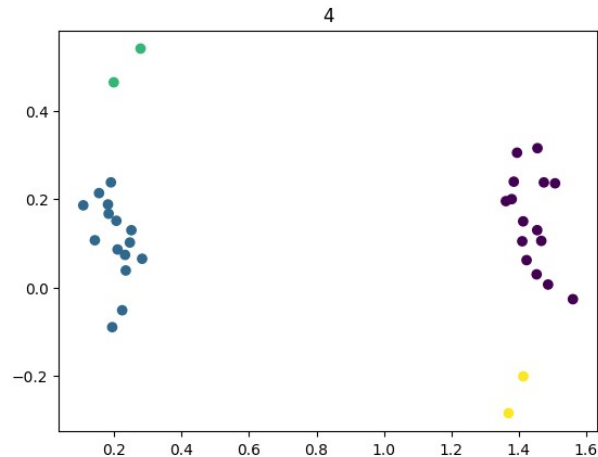
Случайное блуждание – более быстрое решение – негативный саплинг. DeepWalk

```
embedding_dim,lr=3,1e-2
model = GraphEmbedder(num_nodes, embedding_dim).to(device)
optimizer = optim.SGD(model.parameters(), lr=lr)
```

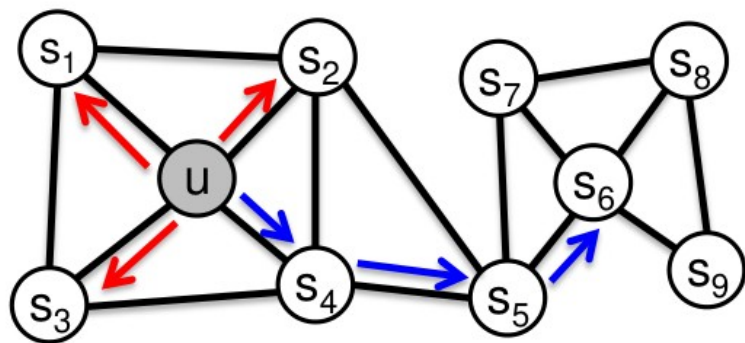
```
walks = generate_random_walks(G, num_walks=1000, walk_length=10)
nodes=list(G.nodes())
batch_size,batch_pairs = generate_training_pairs(nodes, walks)
pairs = np.array(batch_pairs) # (N, 3): [center, context, label]
centers = torch.tensor(pairs[:, 0], dtype=torch.long).to(device)
contexts = torch.tensor(pairs[:, 1], dtype=torch.long).to(device)
labels = torch.tensor(pairs[:, 2], dtype=torch.float).to(device)
```

```
model.train()
num_epochs=10
for epoch in range(num_epochs):
    total_loss = 0.0
    for i in range(0, len(pairs), batch_size):
        batch_centers = centers[i:i+batch_size]
        batch_contexts = contexts[i:i+batch_size]
        batch_labels = labels[i:i+batch_size]
        optimizer.zero_grad()
        center_embeds = model(batch_centers)
        context_embeds = model(batch_contexts)
        scores = torch.sum(center_embeds * context_embeds, dim=1)
        pos_scores = scores[batch_labels == 1]
        neg_scores = scores[batch_labels == 0]
        loss = contrastive_loss(pos_scores,neg_scores)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {total_loss:.4f}")
```

```
model.eval()
with torch.no_grad():
    embeddings = model.embeddings.weight.cpu().data.numpy()
```



Idea: use flexible, biased random walks that can trade off between **local** and **global** views of the network ([Grover and Leskovec, 2016](#)).



→ BFS

→ DFS

Проблема: найти баланс между блужданиями в ширину (BFS) и в глубину (DFS)

Walk of length 3 ($N_R(u)$ of size 3):

$N_{BFS}(u) = \{s_1, s_2, s_3\}$ **Local** microscopic view

$N_{DFS}(u) = \{s_4, s_5, s_6\}$ **Global** macroscopic view

- Two parameters:

- Return parameter p :

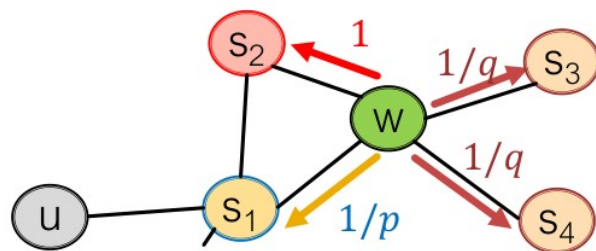
- Return back to the previous node

- In-out parameter q :

- Moving outwards (DFS) vs. inwards (BFS)
- Intuitively, q is the “ratio” of BFS vs. DFS

- Walker came over edge (s_1, w) and is at w .

Where to go next?



Target t	Prob.	Dist. (s_1, t)
s_1	$1/p$	0
s_2	1	1
s_3	$1/q$	2
s_4	$1/q$	2

Unnormalized
transition prob.
segmented based
on distance from s_1

- BFS-like walk: Low value of p

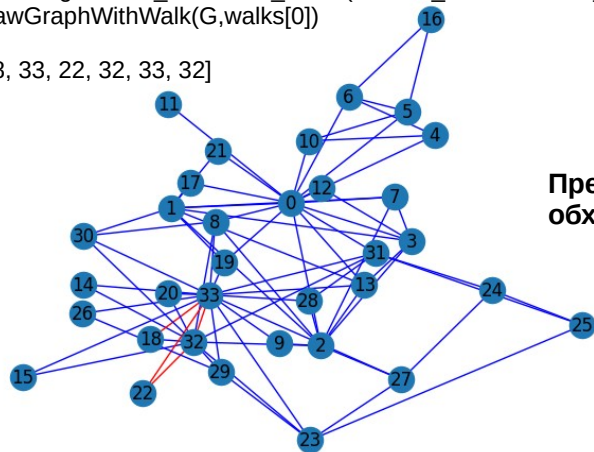
- DFS-like walk: Low value of q

$N_R(u)$ are the nodes visited by the biased walk

Node2Vec

```
walks = generate_node2vec_walks(G, num_walks=1, walk_length=7, p=100., q=100.)  
drawGraphWithWalk(G, walks[0])
```

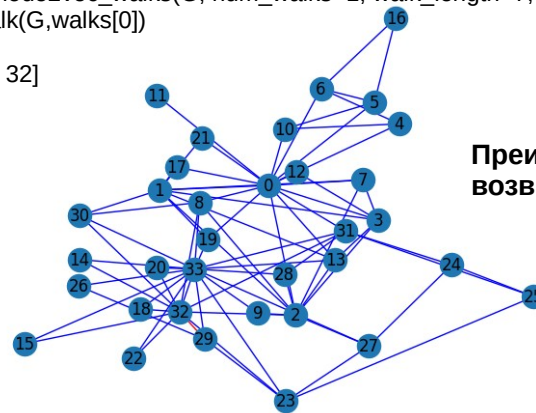
[18, 33, 22, 32, 33, 32]



**Преимущественный
обход в ширину**

```
walks = generate_node2vec_walks(G, num_walks=1, walk_length=7, p=0.001, q=100.)  
drawGraphWithWalk(G, walks[0])
```

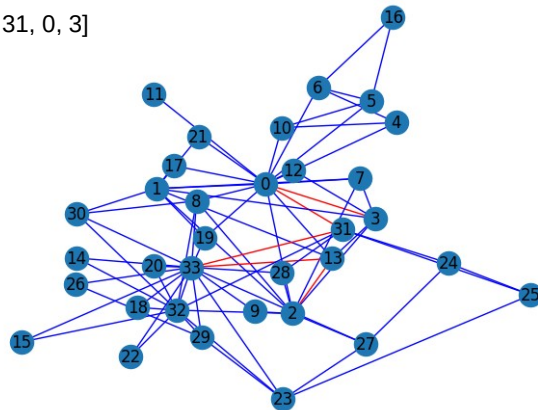
[29, 32, 29, 32, 29, 32]



**Преимущественный
возврат**

```
walks = generate_node2vec_walks(G, num_walks=1, walk_length=7, p=100., q=0.001)  
drawGraphWithWalk(G, walks[0])
```

[2, 13, 33, 31, 0, 3]



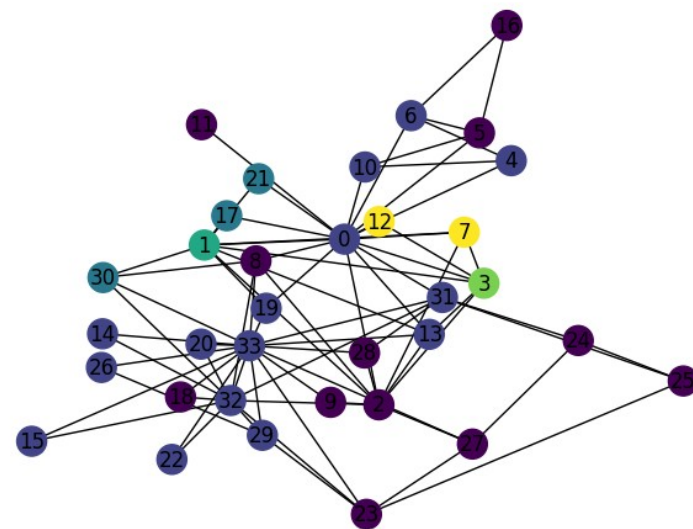
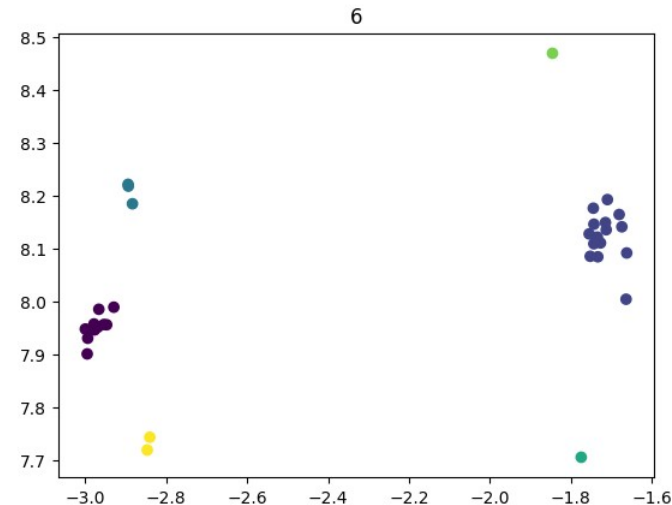
**Преимущественный
обход в глубину**

Node2Vec

```
embedding_dim,lr=3,1e-2
model = GraphEmbedder(num_nodes, embedding_dim).to(device)
optimizer = optim.SGD(model.parameters(), lr=lr)
```

```
walks = generate_node2vec_walks(G, num_walks=100,
walk_length=7,p=1,q=1)
nodes=list(G.nodes())
batch_size,pairs = generate_training_pairs(nodes, walks)
pairs = np.array(pairs) # (N, 3): [center, context, label]
centers = torch.tensor(pairs[:, 0], dtype=torch.long).to(device)
contexts = torch.tensor(pairs[:, 1], dtype=torch.long).to(device)
labels = torch.tensor(pairs[:, 2], dtype=torch.float).to(device)
```

```
model.train()
num_epochs=10
for epoch in range(num_epochs):
    total_loss = 0.0
    for i in range(0, len(pairs), batch_size):
        batch_centers = centers[i:i+batch_size]
        batch_contexts = contexts[i:i+batch_size]
        batch_labels = labels[i:i+batch_size]
        optimizer.zero_grad()
        center_embeds = model(batch_centers) # (B, d)
        context_embeds = model(batch_contexts) # (B, d)
        scores = torch.sum(center_embeds * context_embeds, dim=1)
        pos_scores = scores[batch_labels == 1]
        neg_scores = scores[batch_labels == 0]
        loss = contrastive_loss(pos_scores,neg_scores)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {total_loss:.4f}")
```



Node2Vec via Word2Vec

```
import networkx as nx
import numpy as np
from gensim.models.word2vec import Word2Vec
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

G = nx.karate_club_graph()

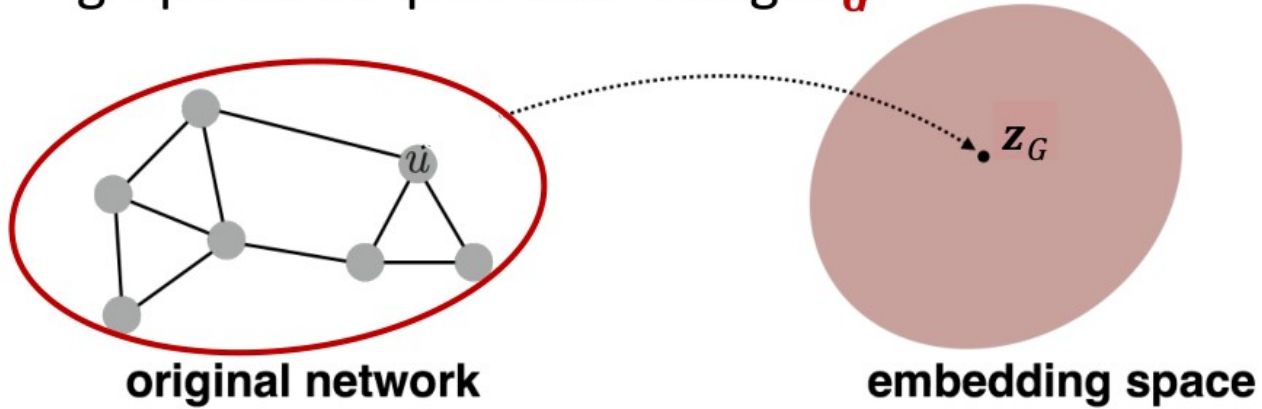
def next_node(G, previous, current, p, q):
    neighbors = list(G.neighbors(current))
    alphas = []
    for neighbor in neighbors:
        if neighbor == previous: alpha = 1/p
        elif G.has_edge(neighbor, previous): alpha = 1
        else: alpha = 1/q
    alphas.append(alpha)
    probs = [alpha / sum(alphas) for alpha in alphas]
    next = np.random.choice(neighbors, size=1, p=probs)[0]
    return next

def random_walk(G, start, length, p, q):
    walk = [start]
    for i in range(length):
        current = walk[-1]
        previous = walk[-2] if len(walk) > 1 else None
        next = next_node(G, previous, current, p, q)
        walk.append(next)
    return [int(x) for x in walk]
```

```
labels = []
for node in G.nodes:
    label = G.nodes[node]['club']
    labels.append(1 if label == 'Officer' else 0)
walks = []
for node in G.nodes:
    for _ in range(80):
        walk = random_walk(G, node, 10, 3, 2)
        walk_str = [str(x) for x in walk]
        walks.append(walk_str)
node2vec = Word2Vec(walks, hs=1, sg=1, vector_size=100,
                    window=10, workers=2, min_count=1, seed=0)
train_mask = list(range(0,34,2))
train_mask_str = [str(x) for x in train_mask]
test_mask = list(range(1,34,2))
test_mask_str = [str(x) for x in test_mask]
labels = np.array(labels)
clf = RandomForestClassifier(random_state=0)
clf.fit(node2vec.wv[train_mask_str], labels[train_mask])
y_pred = clf.predict(node2vec.wv[test_mask_str])
acc = accuracy_score(y_pred, labels[test_mask])
```

Кодировка всего графа

- **Goal:** Want to embed a subgraph or an entire graph G . Graph embedding: $\mathbf{z}_G$.



Подход 1: усреднение по субграфам

Simple (but effective) approach 1:

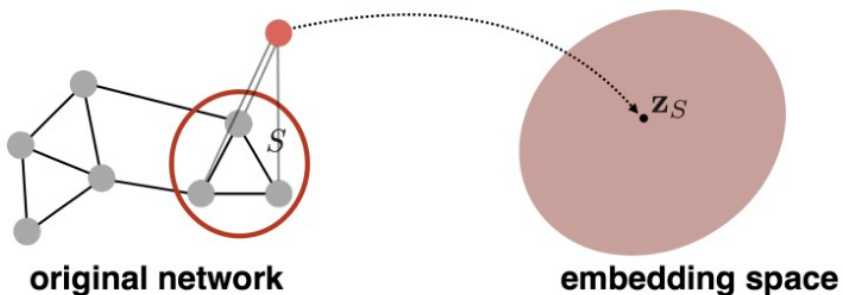
- Run a standard graph embedding technique *on* the (sub)graph G .
- Then just sum (or average) the node embeddings in the (sub)graph G .

$$\mathbf{z}_G = \sum_{v \in G} \mathbf{z}_v$$

```
print(node2vec.wv[[str(x) for x in range(34)]])  
print(node2vec.wv[[str(x) for x in range(34)]].shape)  
print('entire graph:')  
print(node2vec.wv[[str(x) for x in range(34)]].sum(axis=0))
```

Подход 2: виртуальный нод

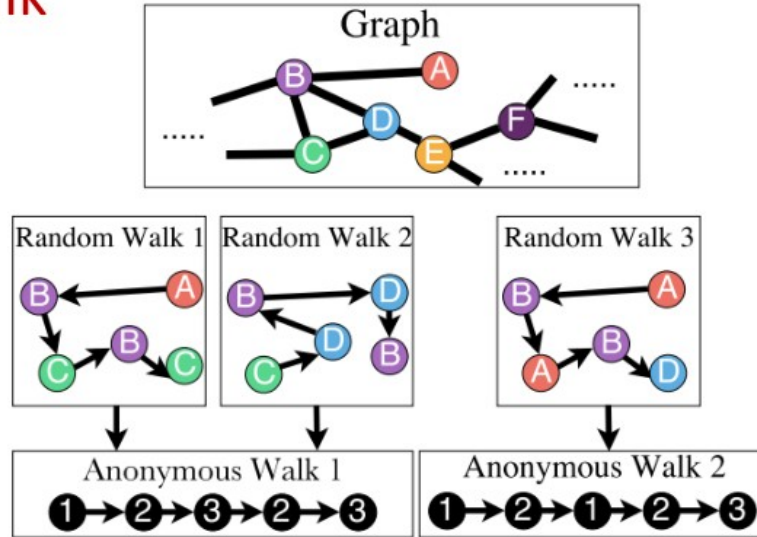
Approach 2: Introduce a “**virtual node**” to represent the (sub)graph and run a standard graph embedding technique



```
Gv=G.copy()
for node in range(34):
    Gv.add_edge(34,node)
walks = []
for node in Gv.nodes:
    for _ in range(80):
        walk = random_walk(Gv,node, 10, 3, 2)
        walk_str = [str(x) for x in walk]
        walks.append(walk_str)
node2vec = Word2Vec(walks, hs=1,
                    sg=1, vector_size=100, window=10,
                    workers=2, min_count=1, seed=0)
node2vec.train(walks,
              total_examples=node2vec.corpus_count, \
                  epochs=30, report_delay=1)
print('virt node emb:',node2vec.wv["34"])
```

Подход 3: анонимные блуждания

States in **anonymous walks** correspond to the index of the **first time** we visited the node in a random walk



- Simulate anonymous walks w_i of l steps and record their counts.
- **Represent the graph as a probability distribution over these walks.**

Подход 3: анонимные блуждания

■ How many random walks m do we need?

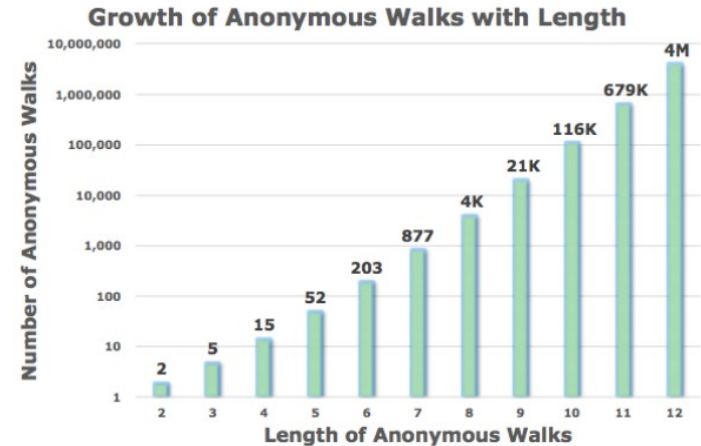
- We want the distribution to have error of more than ε with prob. less than δ :

$$m = \left\lceil \frac{2}{\varepsilon^2} (\log(2^\eta - 2) - \log(\delta)) \right\rceil$$

where: η is the total number of anon. walks of length l .

For example:

There are $\eta = 877$
anonymous walks of length
 $l = 7$. If we set
 $\varepsilon = 0.1$ and $\delta = 0.01$ then
we need to generate
 $m = 122,500$ random walks



Number of anonymous walks grows exponentially

Подход 3: анонимные блуждания

```
def random_anonymous_walk(G, start, length, p, q):
```

```
    walk = [0]
```

```
    encoder, enc_max = {}, 0
```

```
    encoder[start] = 0
```

```
    for i in range(length):
```

```
        current = walk[-1]
```

```
        previous = walk[-2] if len(walk) > 1 else None
```

```
        next = next_node(G, previous, current, p, q)
```

```
        if next not in encoder:
```

```
            enc_max += 1
```

```
            encoder[next] = enc_max
```

```
        walk.append(encoder[next])
```

```
    return [int(x) for x in walk], encoder
```

```
walks = []
```

```
for node in G.nodes:
```

```
    for _ in range(80):
```

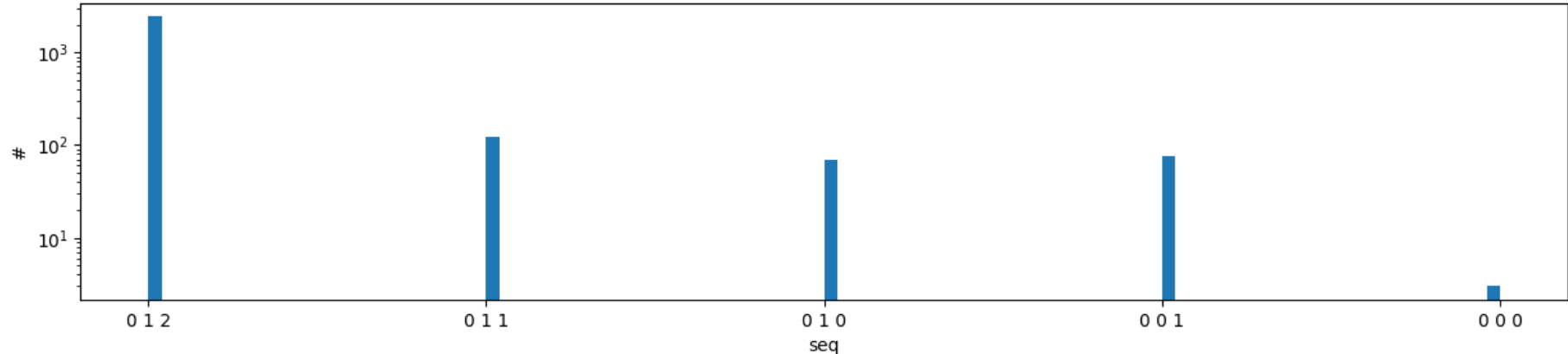
```
        walk, code = random_anonymous_walk(G, node, 2, 3, 2)
```

```
        walk_str = ''.join([str(x) for x in walk])
```

```
        walks.append(walk_str)
```

```
plt.figure(figsize=(15, 3))
```

```
plt.hist(walks, bins=100);
```



ЗАДАНИЕ:

**Сгенерировать 30 различных графов и кластеризовать их.
Интерпретировать результаты кластеризации.**

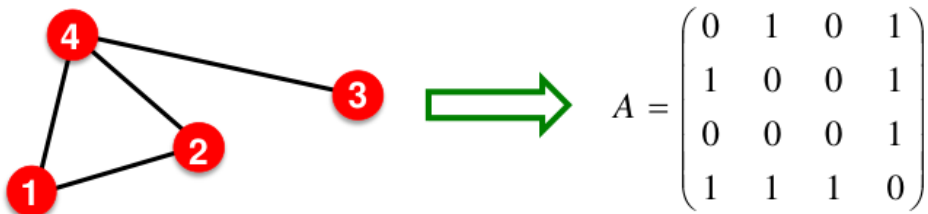
■ How to use embeddings $\mathbf{z}_i$ of nodes:

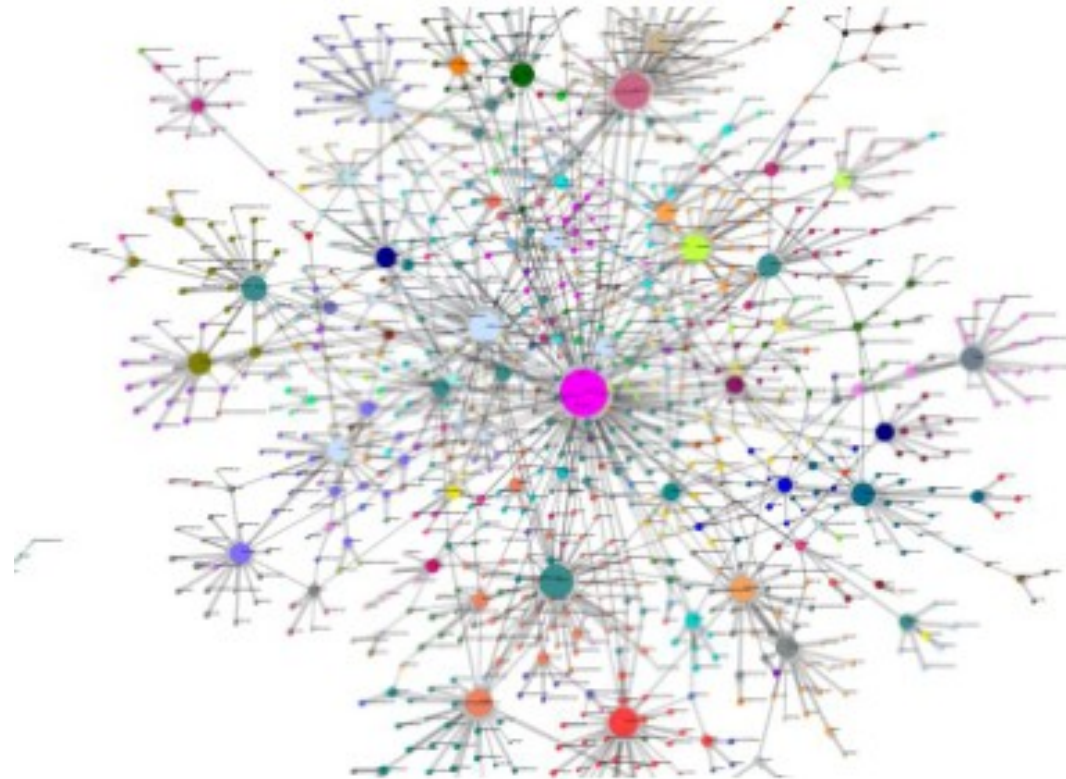
- **Clustering/community detection:** Cluster points $\mathbf{z}_i$
- **Node classification:** Predict label of node i based on $\mathbf{z}_i$
- **Link prediction:** Predict edge (i, j) based on $(\mathbf{z}_i, \mathbf{z}_j)$
 - Where we can: concatenate, avg, product, or take a difference between the embeddings:
 - Concatenate: $f(\mathbf{z}_i, \mathbf{z}_j) = g([\mathbf{z}_i, \mathbf{z}_j])$
 - Hadamard: $f(\mathbf{z}_i, \mathbf{z}_j) = g(\mathbf{z}_i * \mathbf{z}_j)$ (per coordinate product)
 - Sum/Avg: $f(\mathbf{z}_i, \mathbf{z}_j) = g(\mathbf{z}_i + \mathbf{z}_j)$
 - Distance: $f(\mathbf{z}_i, \mathbf{z}_j) = g(\|\mathbf{z}_i - \mathbf{z}_j\|_2)$
- **Graph classification:** Graph embedding $\mathbf{z}_G$ via aggregating node embeddings or anonymous random walks.
Predict label based on graph embedding $\mathbf{z}_G$.

PageRank

(L.Page & S.Brin, Stanford Univ.)

- Treating a graph as a matrix allows us to:
 - Determine node importance via **random walk** (PageRank)
 - Obtain node embeddings via **matrix factorization (MF)**
 - View other **node embeddings** (e.g. Node2Vec) as MF
- **Random walk, matrix factorization and node embeddings are closely related!**





WEB как граф:

- ноды – страницы
- ребра – ссылки

Структура нодов и ребер –
топология сети

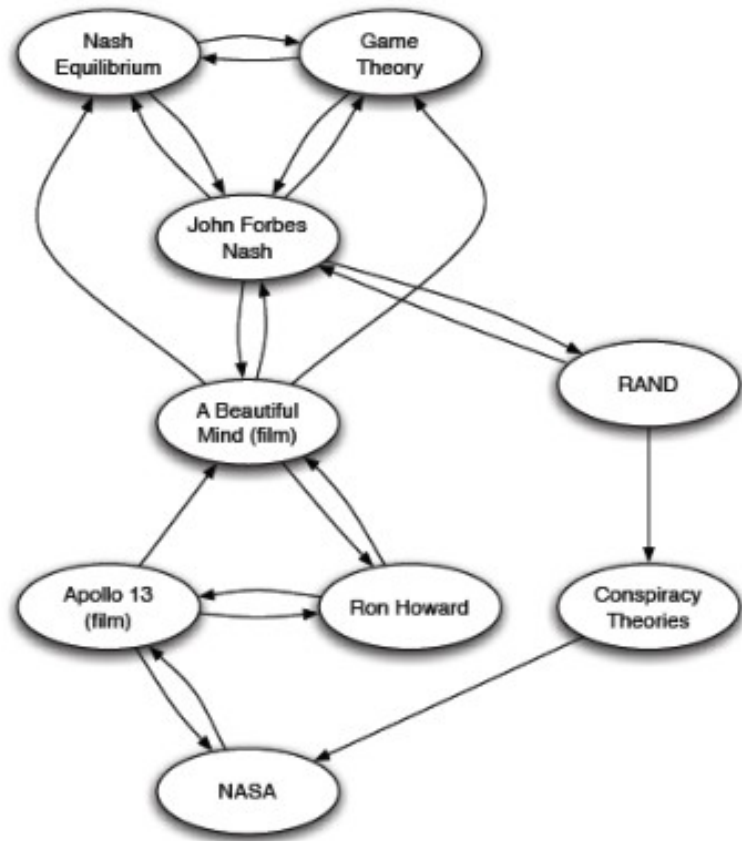
Проблемы графового представления:

- динамические страницы
- защищенные сайты/страницы

Рост типизации связей:

- переход
- оплата
- комментирование
- оценка
- действие

Связи чаще всего однонаправленные



References in an Encyclopedia

- **Idea: Links as votes**
 - Page is more important if it has more links
 - In-coming links? Out-going links?
- **Think of in-links as votes:**
 - www.stanford.edu has 23,400 in-links
 - thispersondoesnotexist.com has 1 in-link
- **Are all in-links equal?**
 - Links from important pages count more
 - Recursive question!

- **A “vote” from an important page is worth more:**
 - Each link’s vote is proportional to the **importance** of its source page
 - If page i with importance r_i has d_i out-links, each link gets r_i / d_i votes
 - Page j ’s own importance r_j is the sum of the votes on its in-links

Ранг:

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

d_i ... out-degree of node i

■ Stochastic adjacency matrix M

- d_i is the outdegree of node i
- If $i \rightarrow j$, then $M_{ji} = \frac{1}{d_i}$
 - M is a **column stochastic matrix**
 - Columns sum to 1

Rank vector r : An entry per page

- r_i is the importance score of page i
- $\sum_i r_i = 1$

$$r = M \cdot r$$

Задача рекурсивная

- The flow equation:

$$1 \cdot \mathbf{r} = \mathbf{M} \cdot \mathbf{r}$$

$$\begin{array}{|c|} \hline \mathbf{r}_y \\ \hline \mathbf{r}_a \\ \hline \mathbf{r}_m \\ \hline \end{array} = \begin{array}{|ccc|} \hline 1/2 & 1/2 & 0 \\ \hline 1/2 & 0 & 1 \\ \hline 0 & 1/2 & 0 \\ \hline \end{array} \begin{array}{|c|} \hline \mathbf{r}_y \\ \hline \mathbf{r}_a \\ \hline \mathbf{r}_m \\ \hline \end{array}$$

$\mathbf{r} \qquad \mathbf{M} \qquad \mathbf{r}$

- So the **rank vector** $\mathbf{r}$ is an **eigenvector** of the stochastic adj. matrix $\mathbf{M}$ (with eigenvalue 1)
 - Starting from any vector $\mathbf{u}$, the limit $\mathbf{M}(\mathbf{M}(\dots \mathbf{M}(\mathbf{M} \mathbf{u})))$ is the **long-term distribution** of the surfers.
 - **PageRank** = Limiting distribution = **principal eigenvector** of $\mathbf{M}$
 - **Note**: If $\mathbf{r}$ is the limit of the product $\mathbf{M}\mathbf{M} \dots \mathbf{M}\mathbf{u}$, then $\mathbf{r}$ satisfies the **flow equation** $1 \cdot \mathbf{r} = \mathbf{M}\mathbf{r}$
 - So $\mathbf{r}$ is the **principal eigenvector** of $\mathbf{M}$ with eigenvalue 1
- **We can now efficiently solve for $\mathbf{r}$!**
 - The method is called **Power iteration**

Given a graph with n nodes, we use an iterative procedure:

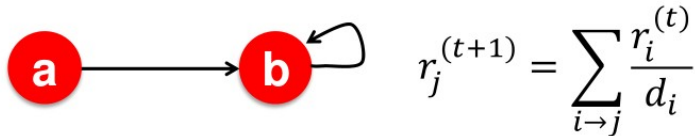
- Assign each node an initial page rank
- Repeat until convergence ($\sum_i |r_i^{t+1} - r_i^t| < \epsilon$)
 - Calculate the page rank of each node

$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

d_i out-degree of node i

- **(1)** Some pages are **dead ends** (have no out-links)
 - Such pages cause importance to “leak out”
- **(2) Spider traps**
(all out-links are within the group)
 - Eventually spider traps absorb all importance

■ The “Spider trap” problem:



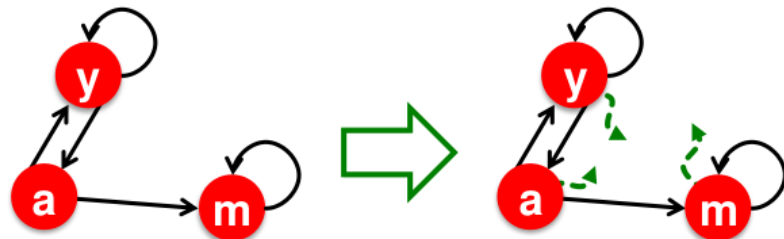
■ Example:

Iteration:	0,	1,	2,	3...
r_a	1	0	0	0
r_b	0	1	1	1

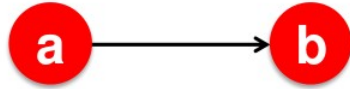
■ **Solution for spider traps: At each time step, the random surfer has two options**

- With prob. β , follow a link at random
- With prob. $1-\beta$, jump to a random page
- Common values for β are in the range 0.8 to 0.9

■ **Surfer will teleport out of spider trap within a few time steps**



■ The “Dead end” problem:

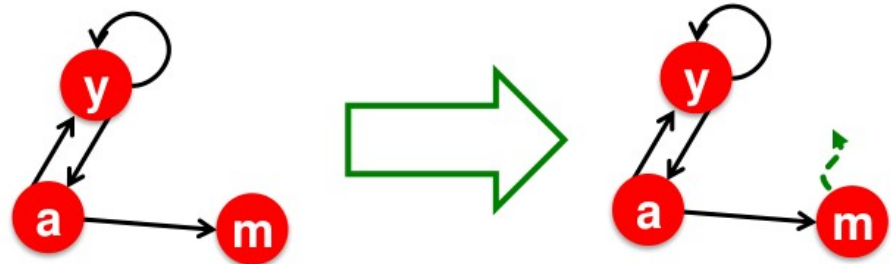


$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

■ Example:

Iteration:	0,	1,	2,	3...
r_a	1	0	0	0
r_b	0	1	0	0

- **Teleports:** Follow random teleport links with total probability **1.0** from dead-ends
 - Adjust matrix accordingly



- Google's solution that does it all:

At each step, random surfer has two options:

- With probability β , follow a link at random
- With probability $1-\beta$, jump to some random page

- **PageRank equation** [Brin-Page, 98]

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$

$d_i \dots$ out-degree of node i

Возможная корректура на тупики:

$$r_j = \sum_{i \rightarrow j} \beta \frac{d_i r_i}{(d_i + \epsilon)^2} + (1 - \beta) \frac{1}{N}$$

- **The Google Matrix G :**

$[1/N]_{N \times N} \dots N$ by N matrix where all entries are $1/N$

$$G = \beta M + (1 - \beta) \left[\frac{1}{N} \right]_{N \times N}$$

- **We have a recursive problem: $\mathbf{r} = \mathbf{G} \cdot \mathbf{r}$**

And the Power method still works!

- **What is β ?**

- In practice $\beta = 0.8, 0.9$ (make 5 steps on avg., jump)

```

import numpy as np
M=np.array([[0.5,0.5,0],[0.5,0,0],[0,0.5,1]])
Nm=1/3*np.array([[1,1,1],[1,1,1],[1,1,1]])
b=0.8
G=b*M+(1-b)*Nm
for i in range(10):
    Gold=G.copy()
    G=G@G
    print("\nitt:',i,'\nres:\n',G)
    if ((np.abs(G-Gold)<1e-10).all()):
        print('converged')
        break

```

```

itt: 0
[[0.44    0.28    0.12   ]
 [0.25333333 0.25333333 0.09333333]
 [0.30666667 0.46666667 0.78666667]]

```

```

itt: 1
[[0.30133333 0.25013333 0.17333333]
 [0.20426667 0.17866667 0.12746667]
 [0.4944    0.5712    0.6992   ]]

```

```

itt: 2
[[0.22759168 0.219072  0.20530944]
 [0.16106752 0.15582464 0.14730496]
 [0.6113408  0.62510336 0.6473856  ]]

```

```

itt: 3
[[0.21259739 0.2123354  0.21191149]
 [0.15180945 0.15164753 0.15138553]
 [0.63559316 0.63601707 0.63670298]]

```

```

itt: 4
[[0.21212166 0.21212142 0.21212101]
 [0.15151543 0.15151528 0.15151503]
 [0.63636291 0.63636331 0.63636396]]

```

```

itt: 5
[[0.21212121 0.21212121 0.21212121]
 [0.15151515 0.15151515 0.15151515]
 [0.63636364 0.63636364 0.63636364]]

```

```

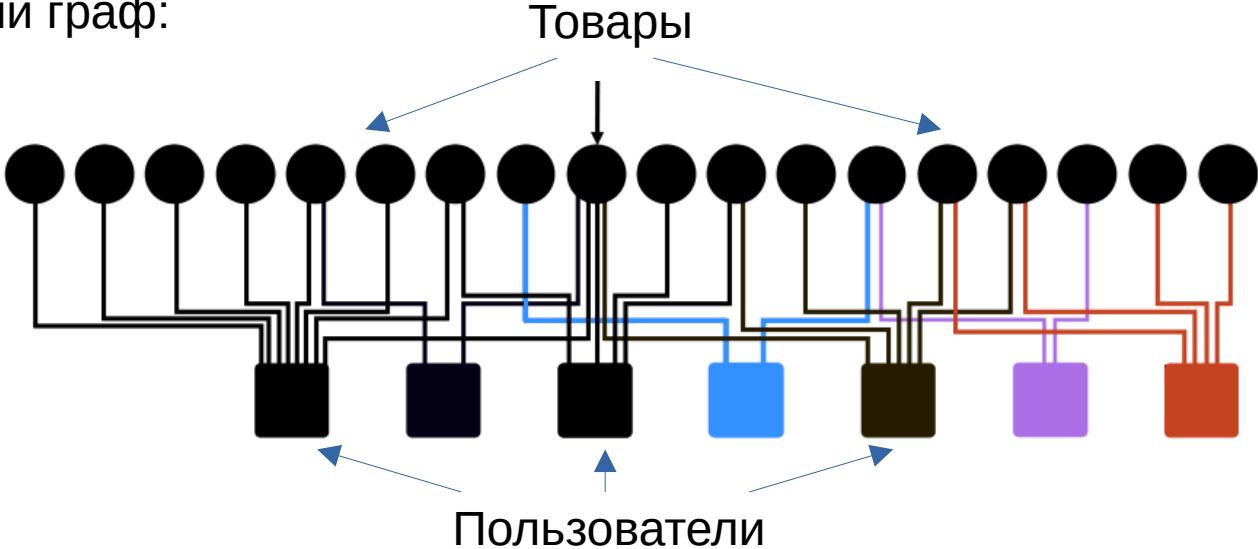
itt: 6
[[0.21212121 0.21212121 0.21212121]
 [0.15151515 0.15151515 0.15151515]
 [0.63636364 0.63636364 0.63636364]]

```

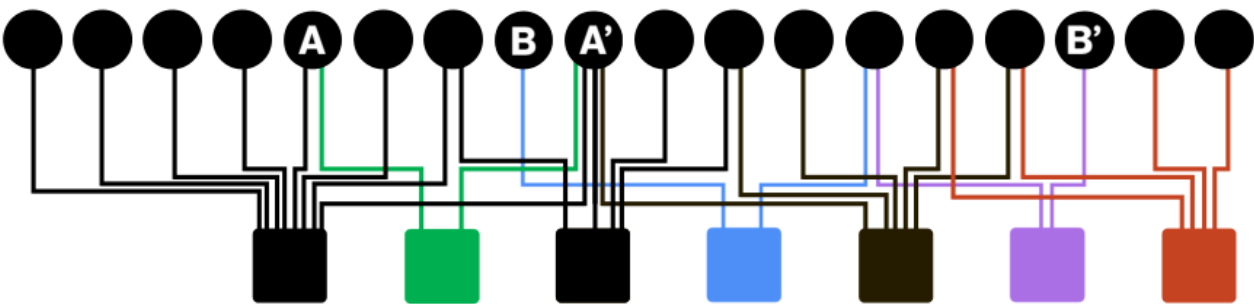
converged

Проблема рекомендаций

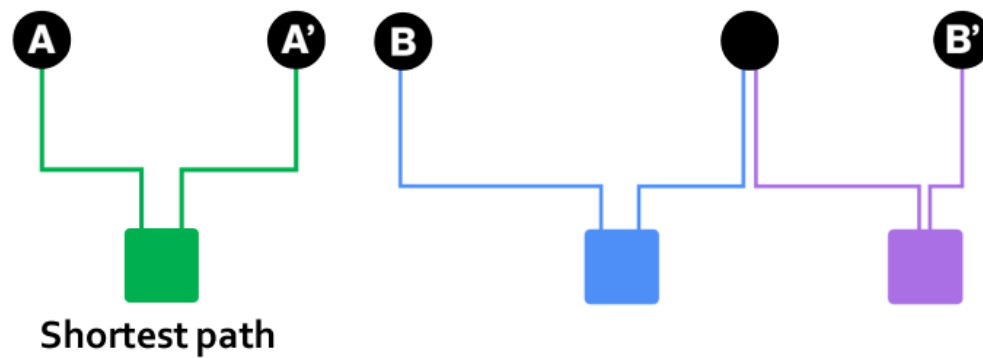
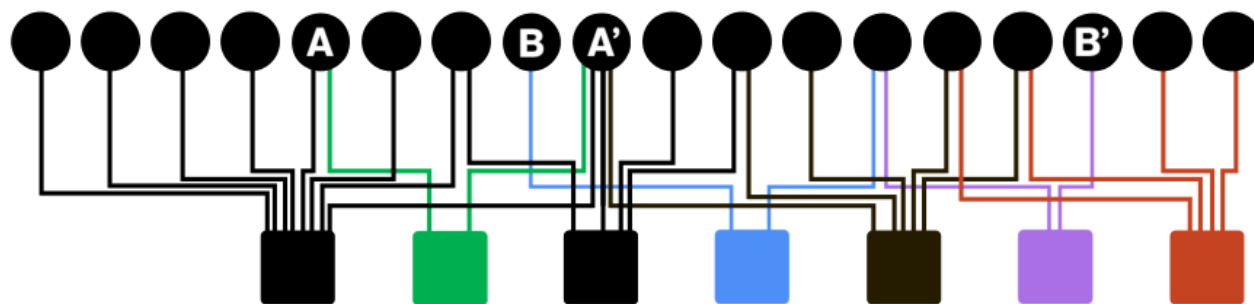
Двудольный граф:



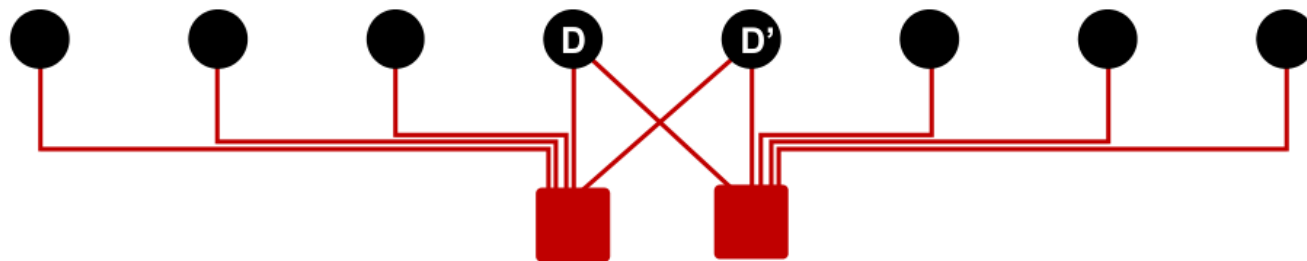
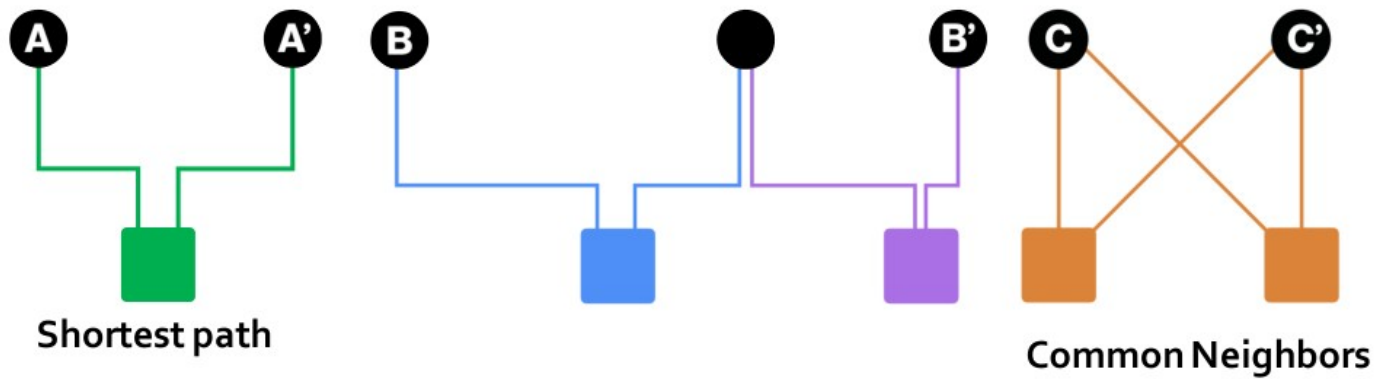
Что ближе – А и А' или В и В' ?



Что ближе – А и А' или В и В' ?



Что ближе – A и A' или B и B' или C и C' или D и D'?

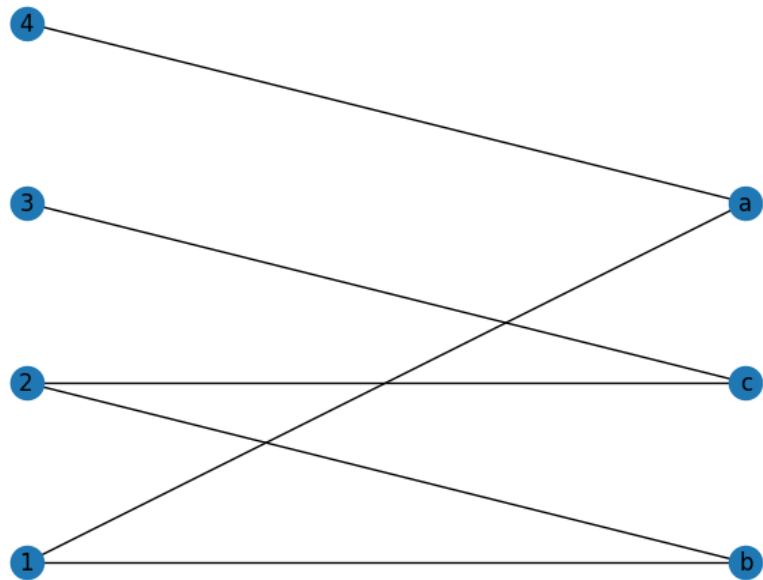


```
import networkx as nx
import matplotlib.pyplot as plt
B = nx.Graph()
B.add_nodes_from([1,2,3,4], bipartite=0) # Add the node attribute "bipartite"
B.add_nodes_from(['a','b','c'], bipartite=1)
B.add_edges_from([(1,'a'), (1,'b'), (2,'b'), (2,'c'), (3,'c'), (4,'a')])
```

```
l, r = nx.bipartite.sets(B)
pos = {}
```

```
pos.update((node, (1, index)) for index, node in enumerate(l))
pos.update((node, (2, index)) for index, node in enumerate(r))
```

```
nx.draw(B, pos=pos, with_labels=True)
plt.show()
```



■ Proximity to query node(s) Q :

$\text{ALPHA} = 0.5$

$\text{QUERY_NODES} = \{ \text{Q} \}$

```
item = QUERY_NODES.sample_by_weight()
```

```
for i in range( N_STEPS ):
```

```
    user = item.get_random_neighbor()
```

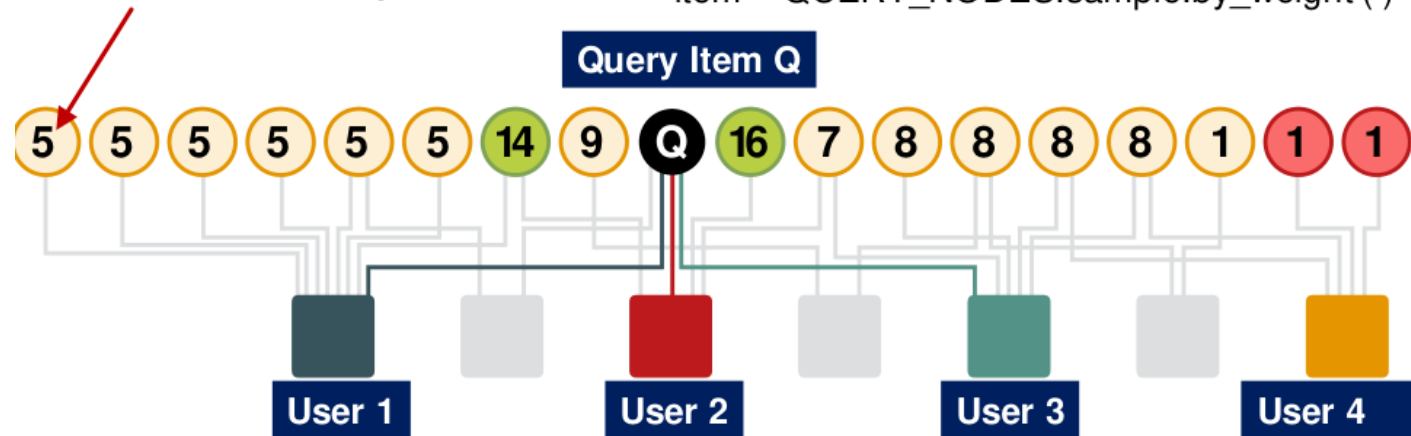
```
    item = user.get_random_neighbor()
```

```
    item.visit_count += 1
```

```
    if random() < ALPHA:
```

```
        item = QUERY_NODES.sample_by_weight()
```

Number of visits by
random walks starting at Q



```

UIDS=np.unique(ratings.user_id)
MIDS=np.unique(ratings.movie_id)
B = nx.Graph()
for x in list(UIDS): B.add_node('u'+str(x),weight=0, bipartite=1)
for x in list(MIDS): B.add_node('m'+str(x),weight=0, bipartite=0)
for i in range(ratings.shape[0]):
    if np.random.rand()<0.01:
        B.add_edge('m'+str(ratings.iloc[i].movie_id),'u'+str(ratings.iloc[i].user_id))
l=['m'+str(x) for x in list(MIDS)]
r=['u'+str(x) for x in list(UIDS)]
pos = {}
pos.update((node, (1, index)) for index, node in enumerate(l))
pos.update((node, (2, index)) for index, node in enumerate(r))
nx.draw(B, pos=pos,node_size=1)
plt.show()

```

```

alpha=0.1
for Q in B.nodes:
    q=Q
    for i in range(100):
        ns=list(B.neighbors(q))
        if ns:
            item=np.random.choice(ns)
            ns=list(B.neighbors(item))
            if ns:
                q=np.random.choice(ns)
                B.nodes[q]['weight']+=1
                if np.random.rand()<alpha:
                    q=Q
    break
for Q in B.nodes:
    if B.nodes[Q]['bipartite']==0:
        print(Q,B.nodes[Q]['weight'])

```

```

m1 11
m2 0
m3 0
m4 7
m5 0
m6 0
m7 10
m8 16
m9 10
m10 0
m11 7
m12 23
m13 0
m14 11
m15 10
m16 0
m17 11
m18 0
m19 0
m20 7
m21 0
m22 11
m23 0
m24 10
m25 10
m66 16
...

```

Варианты PageRank

■ PageRank:

- Teleports to any node
- Nodes can have the same probability of the surfer landing:

$$\mathbf{S} = [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]$$

■ Topic-Specific PageRank aka Personalized PageRank:

- Teleports to a specific set of nodes
- Nodes can have different probabilities of the surfer landing there:

$$\mathbf{S} = [0.1, 0, 0, 0.2, 0, 0, 0.5, 0, 0, 0.2]$$

■ Random Walk with Restarts:

- Topic-Specific PageRank where teleport is always to the same node:

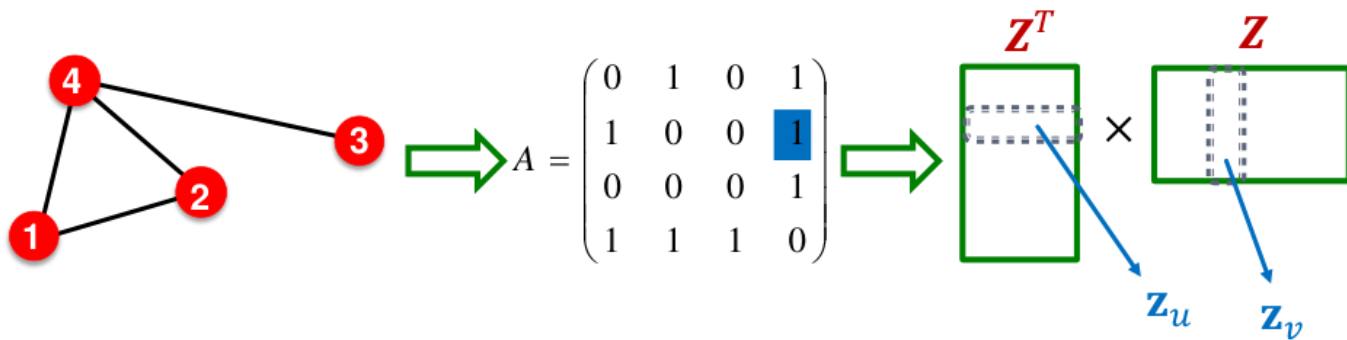
$$\mathbf{S} = [0, 0, 0, 0, \mathbf{1}, 0, 0, 0, 0, 0]$$

Все эти алгоритмы – одинаковы, просто разные матрицы $\mathbf{N}=\mathbf{S}$:

$$r_j = \sum_i (\beta M_{j,i} r_i + (1 - \beta) N_{j,i})$$

Связь задачи поиска близких нод с факторизацией матриц (мелкий эмбединг)

- Simplest **node similarity**: Nodes u, v are similar if they are connected by an edge
- This means: $\mathbf{z}_v^T \mathbf{z}_u = A_{u,v}$
which is the (u, v) entry of the graph adjacency matrix A
- Therefore, **$\mathbf{Z}^T \mathbf{Z} = A$**



Связь задачи поиска близких нод с факторизацией матриц (мелкий эмбединг)

- The embedding dimension d (number of rows in $\mathbf{Z}$) is much smaller than number of nodes n .
- Exact factorization $\mathbf{A} = \mathbf{Z}^T \mathbf{Z}$ is generally not possible
- However, we can learn $\mathbf{Z}$ approximately

■ **Objective:** $\min_{\mathbf{Z}} \|\mathbf{A} - \mathbf{Z}^T \mathbf{Z}\|_2$

- We optimize $\mathbf{Z}$ such that it minimizes the L2 norm (Frobenius norm) of $\mathbf{A} - \mathbf{Z}^T \mathbf{Z}$
- Note in Lecture 3 we used softmax instead of L2. But the goal to approximate $\mathbf{A}$ with $\mathbf{Z}^T \mathbf{Z}$ is the same.

- Conclusion: **Inner product decoder with node similarity defined by edge connectivity is equivalent to matrix factorization of $\mathbf{A}$.**

(эквивалентно для невзвешанных графов)

Эквивалентно при $u \sim v$: $\mathbf{Z}_u^T \mathbf{Z}_v \rightarrow \max$,

При $u \not\sim v$ $\mathbf{Z}_u^T \mathbf{Z}_v \rightarrow \min$,

$$P(v|\mathbf{z}_u) = \frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)} \rightarrow \max$$

Связь задачи поиска близких нод с факторизацией матриц (мелкий эмбединг)

Разложение по Холецкому

```
import numpy as np
A = np.array([[1, 1, 0.1 ], [1,2,1], [0.1, 1, 2]])
print(A)
B=np.linalg.cholesky(A)
print(B)
print(B@B.T)
```

Неотрицательная матричная факторизация

```
import numpy as np
from sklearn.decomposition import NMF

model = NMF(n_components=2, init='random', random_state=0)
W = model.fit_transform(A)
HT = model.components_
B=W
print("A:\n",A)
print("B:\n",B)
print("H:\n",HT)

print("B BT:\n",B @ B.T)
print("W HT:\n", W @ HT)
```

Разложение через собственные вектора

```
import numpy as np
from scipy.linalg import eigh

A = np.array([[0.1, 1, 0.1 ], [1,2,1], [0.1, 1, 2]])
eigenvalues, eigenvectors = eigh(A)
eigenvalues = np.maximum(eigenvalues, 0)
sqrt_eigenvalues = np.sqrt(eigenvalues)
B = eigenvectors @ np.diag(sqrt_eigenvalues)

print("A:\n", A,'\n')
print("B:\n", np.round(B,2),'\n')
print("B @ B.T:\n", B @ B.T,'\n')
```

Связь задачи поиска близких нод с факторизацией матриц (случайные блуждания)

Кодирование случайными блужданиями тоже может быть формализована как разложение матрицы, но другого вида:

Volume of graph
 $vol(G) = \sum_i \sum_j A_{i,j}$

Diagonal matrix D
 $D_{u,u} = \deg(u)$

context window size
See Lec 3 slide 30:
 $T = |N_R(u)|$

Power of normalized adjacency matrix

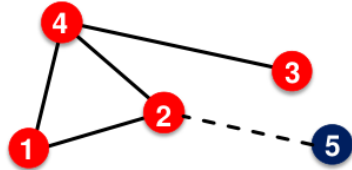
Number of negative samples

$$M = \log \left(vol(G) \left(\frac{1}{T} \sum_{r=1}^T (D^{-1}A)^r \right) D^{-1} \right) - \log b \longrightarrow Z_u^T Z_v$$

Задание: матрица из предыдущего задания, $b=0.1$, написать матрицу M , получить матрицу эмбедингов Z методом собственных значений

Ограничения факторизации

- Cannot obtain embeddings for nodes not in the training set

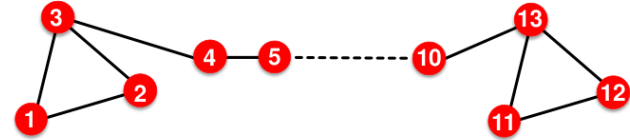


Training set

A newly added node 5 at test time
(e.g., new user in a social network)

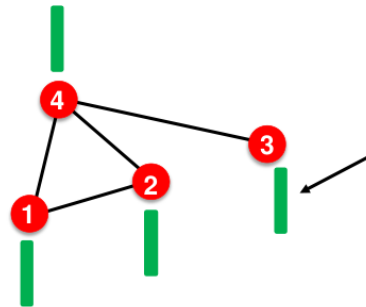
Cannot compute its embedding
with DeepWalk / node2vec. Need to
recompute all node embeddings.

- Cannot capture **structural similarity**:



- Node 1 and 11 are **structurally similar** – part of one triangle, degree 2, ...
- However, they have very **different** embeddings.
 - It's unlikely that a random walk will reach node 11 from node 1.
- DeepWalk and node2vec do not capture structural similarity.**

- Cannot utilize node, edge and graph features



Feature vector
(e.g. protein properties in a
protein-protein interaction graph)

DeepWalk / node2vec
embeddings do not incorporate
such node features

**Solution to these limitations: Deep Representation
Learning and Graph Neural Networks**